Binary Logistic Regression

</> Code

On this page
Introduction
Linear Probability Model
Binary Logistic Regression Model
Testing Assumptions
Predicted Values

Introduction

These notes will primary focus on binary logistic regression. It is the most common type of logistic regression, and sets up the foundation for both ordinal and nominal logistic regression.

The linear probability model is not as widely used since probabilities do not tend to follow the properties of linearity in relation to their predictors. Also, the linear probability model possibly produces predictions outside of the bounds of 0 and 1 (where probabilities should be!). For completeness sake however, here is the linear probability model:

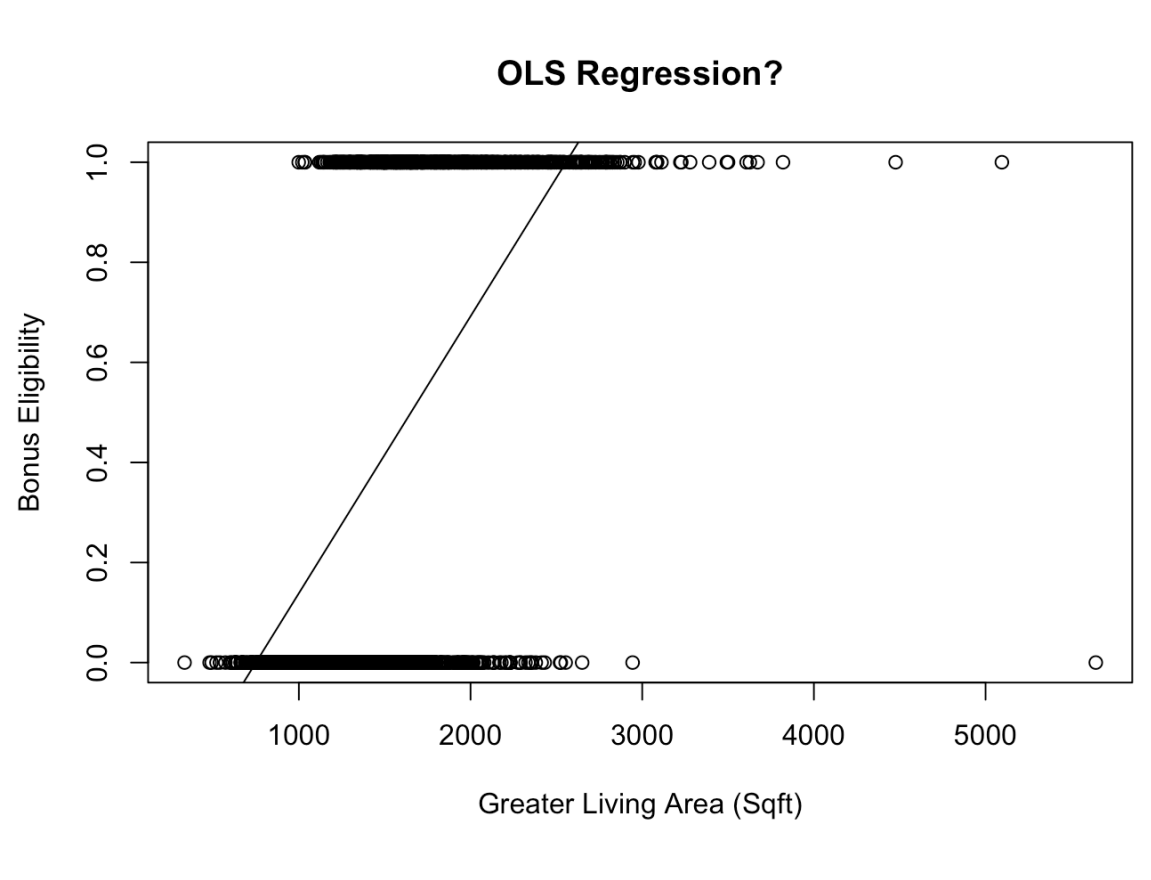
Linear Probability Model

Let's first view what a linear probability model would look like plotted on our data and then we can build the model.

R Python SAS

▼ Code

```
lp.model <- lm(Bonus ~ Gr_Liv_Area, data = train)

with(train, plot(x = Gr_Liv_Area, y = Bonus,
                 main = 'OLS Regression?',
                 xlab = 'Greater Living Area (Sqft)',
                 ylab = 'Bonus Eligibility'))
abline(lp.model)
```



Even though it doesn't appear to really look like our data, let's fit this linear probability model anyway for completeness sake.

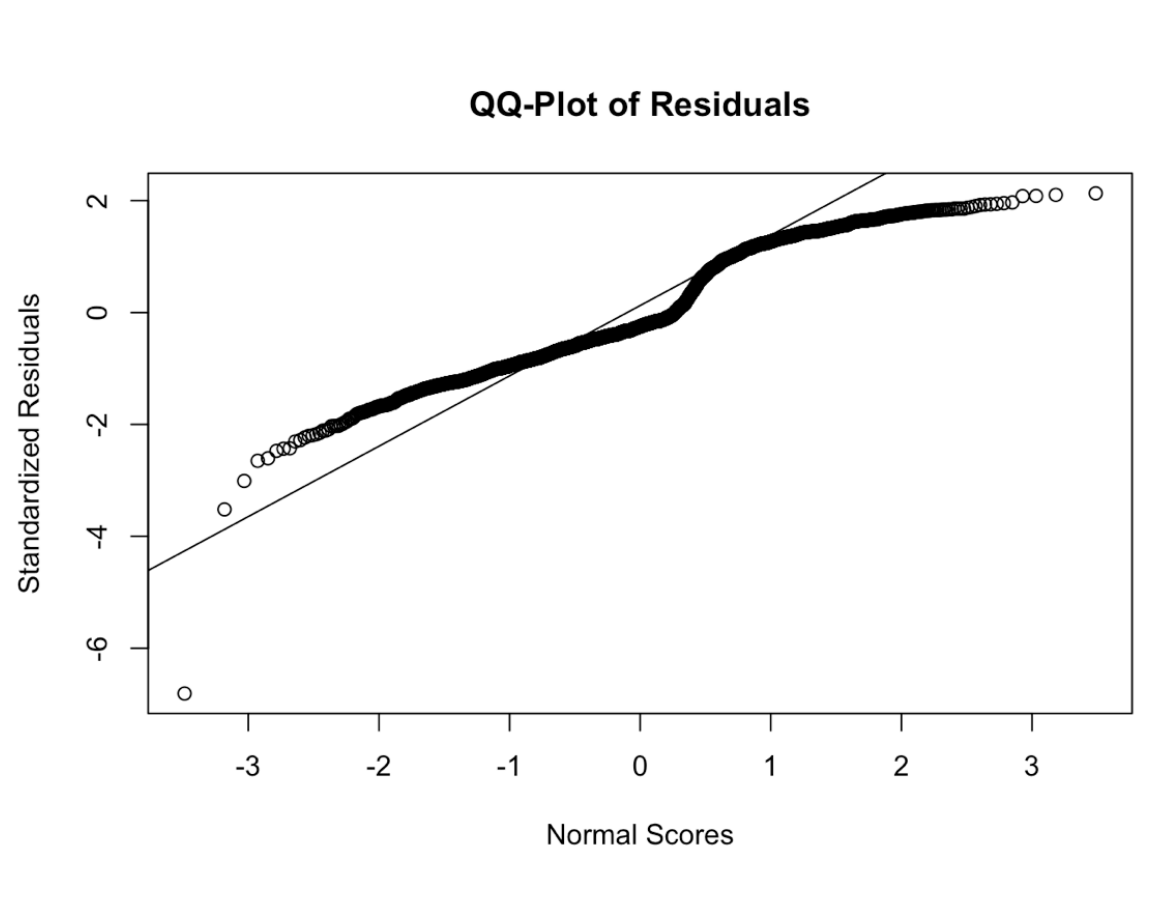
▼ Code

```
lp.model <- lm(Bonus ~ Gr_Liv_Area, data = train)

summary(lp.model)
```

```
Call:
lm(formula = Bonus ~ Gr_Liv_Area, data = train)

Residuals:
     Min       1Q   Median       3Q      Max 
-2.70766 -0.29160 -0.09983  0.39432  0.86198 

Coefficients:
              Estimate Std. Error t value Pr(>|t|)    
(Intercept) -4.149e-01  2.776e-02  -14.94   <2e-16 ***
Gr_Liv_Area  5.534e-04  1.765e-05   31.36   <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.4044 on 2049 degrees of freedom
Multiple R-squared:  0.3243,	Adjusted R-squared:  0.324 
F-statistic: 983.6 on 1 and 2049 DF,  p-value: < 2.2e-16
```

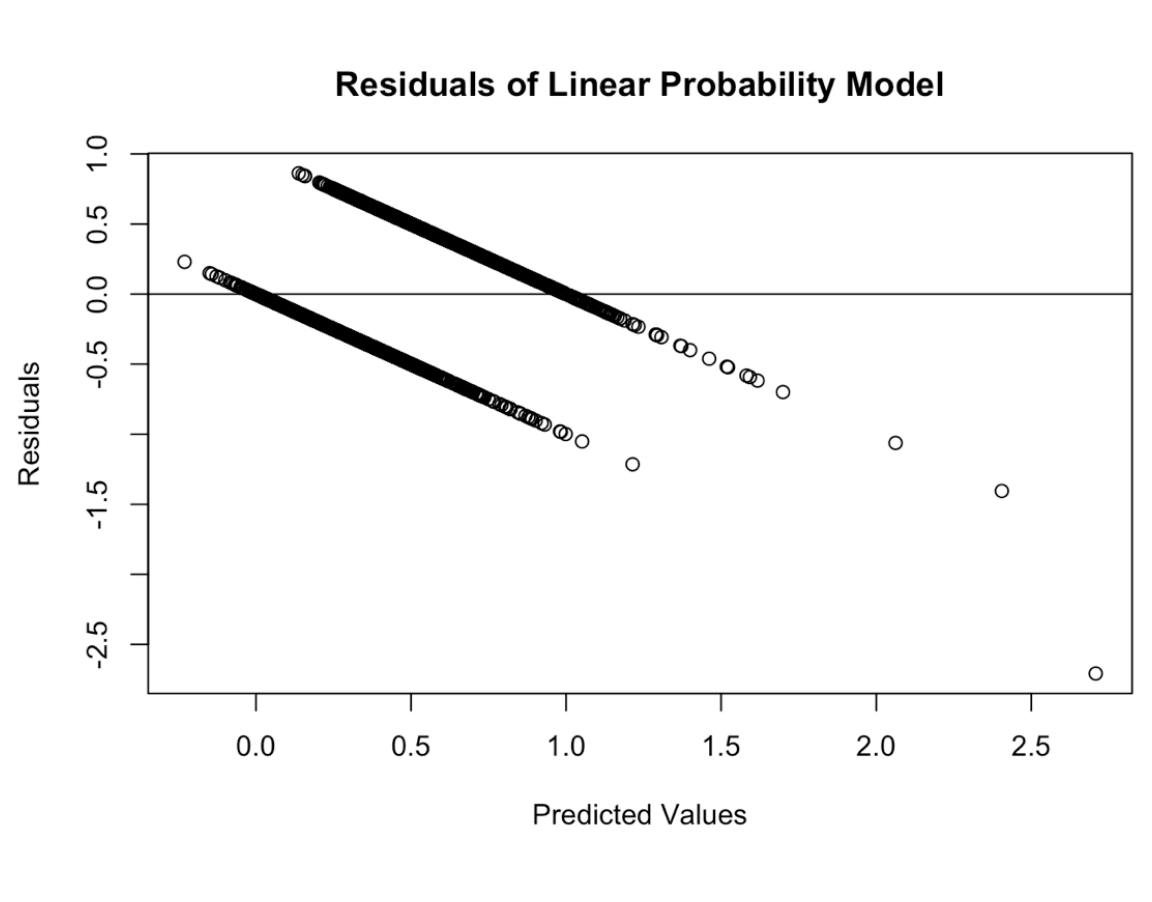
▼ Code

```
qqnorm(rstandard(lp.model),
       ylab = "Standardized Residuals",
       xlab = "Normal Scores",
       main = "QQ-Plot of Residuals")
qqline(rstandard(lp.model))
```



▼ Code

```
plot(predict(lp.model), resid(lp.model),
     ylab="Residuals", xlab="Predicted Values",
     main="Residuals of Linear Probability Model")
abline(0, 0)
```



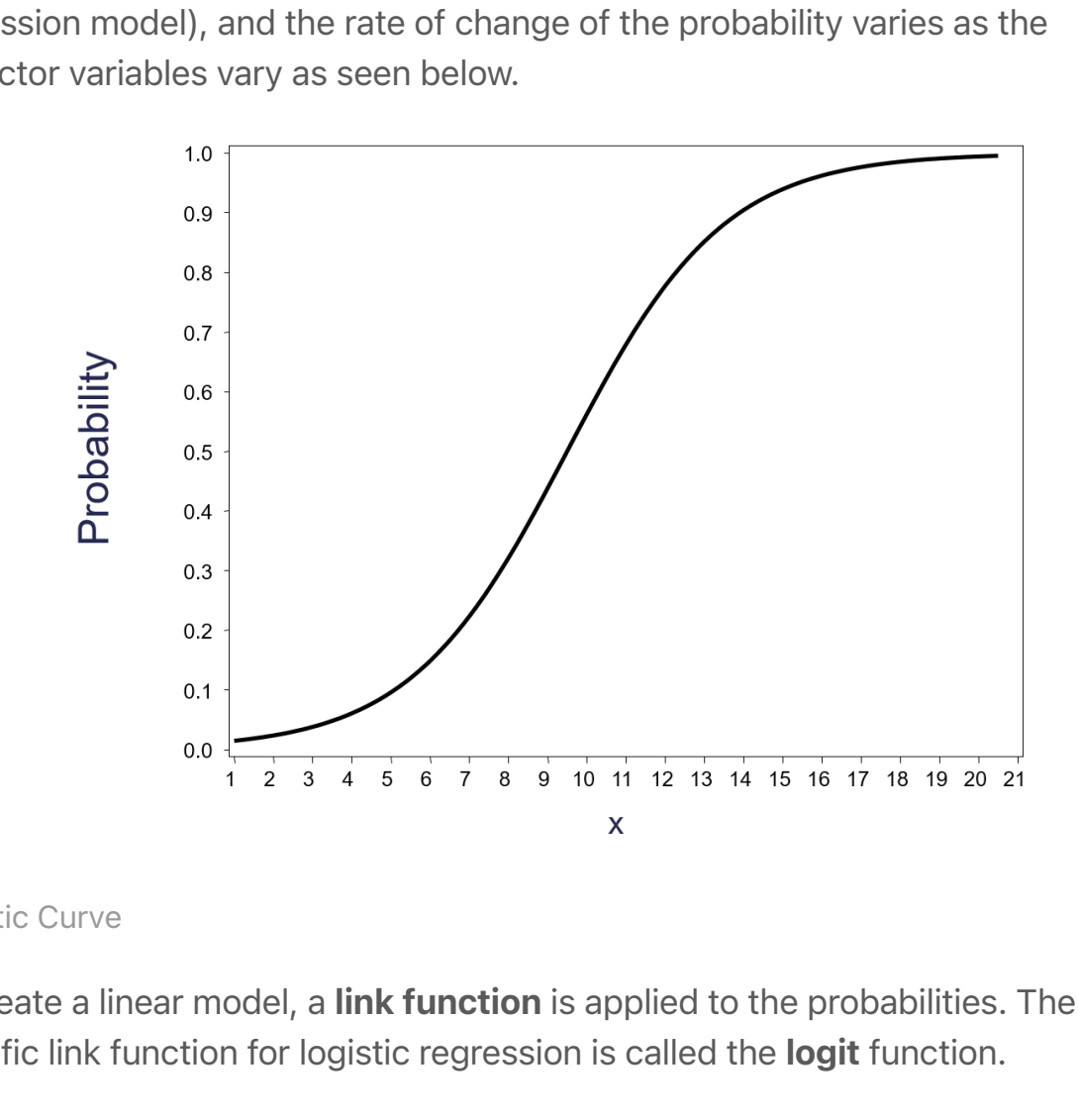
As we can see from the charts above, the assumptions of ordinary least squares don't really hold in this situation. Therefore, we should be careful interpreting the results of the model. Maybe a better model won't have these problems?

Binary Logistic Regression Model

Due to the limitations of the linear probability model, people typically just use the binary logistic regression model. The logistic regression model does not have the limitations of the linear probability model. The outcome of the logistic regression model is the probability of getting a 1 in a binary variable, $E(y_i) = P(y_i = 1) = p_i$. That probability is calculated as follows:

$$p_i = \frac{1}{1 + e^{-(\beta_0 + \beta_1 x_{1,i} + \cdots + \beta_k x_{k,i})}}$$

This function has the desired properties for predicting probabilities. The predicted probability from the above equation will always be between 0 and 1. The parameter estimates do not enter the function linearly (this is a non-linear regression model), and the rate of change of the probability varies as the predictor variables vary as seen below.



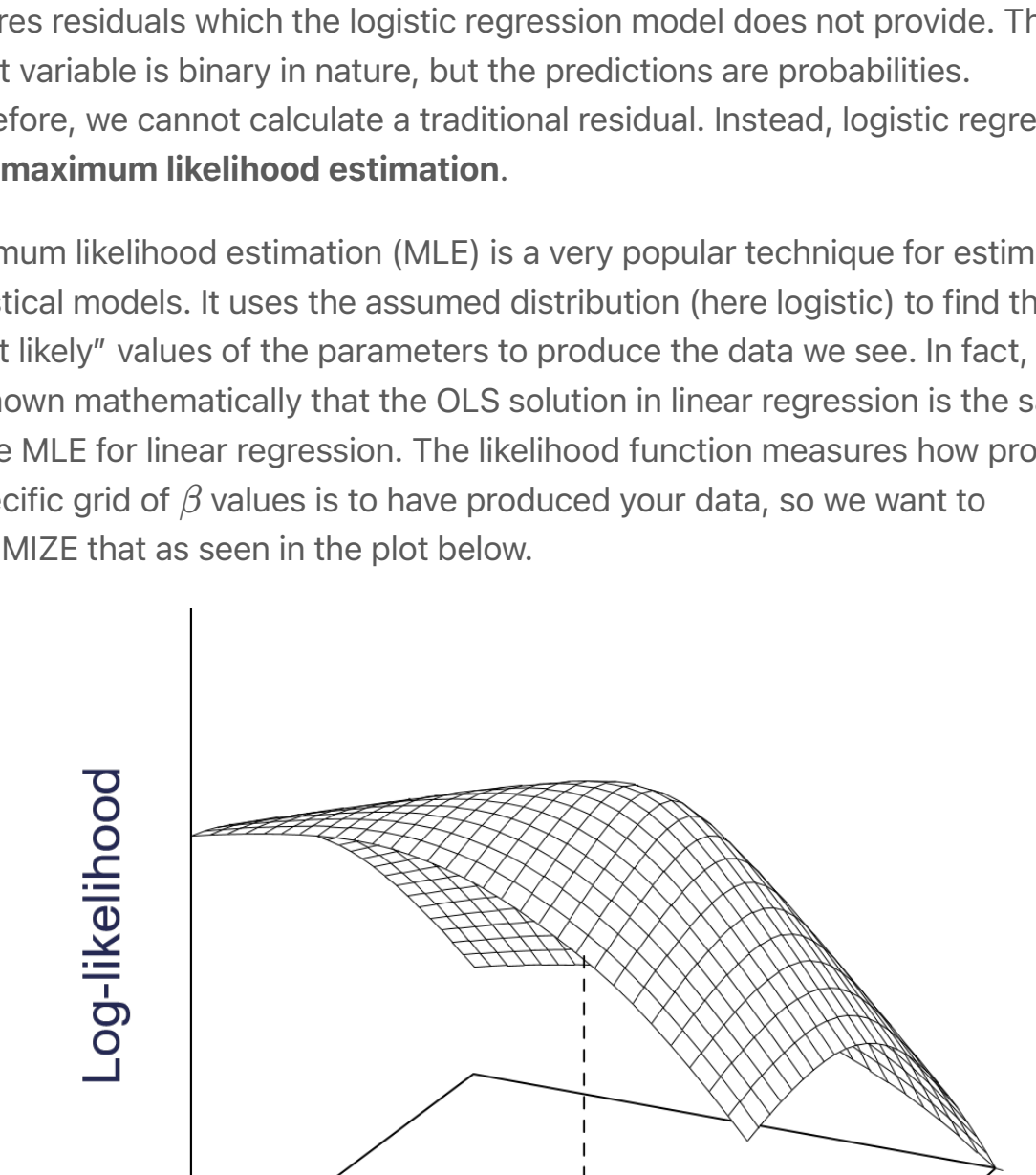
Logistic Curve

To create a linear model, a **link function** is applied to the probabilities. The specific link function for logistic regression is called the **logit** function.

$$logit(p_i) = \log(\frac{p_i}{1 - p_i}) = \beta_0 + \beta_1 x_{1,i} + \cdots + \beta_k x_{k,i}$$

The relationship between the predictor variables and the logits are linear in nature as the logits themselves are unbounded. This structure looks much more like our linear regression model structure. However, logistic regression does not use ordinary least squares (OLS) to estimate the coefficients in our model. OLS requires residuals which the logistic regression model does not provide. The target variable is binary in nature, but the predictions are probabilities. Therefore, we cannot calculate a traditional residual. Instead, logistic regression uses **maximum likelihood estimation**.

Maximum likelihood estimation (MLE) is a very popular technique for estimating statistical models. It uses the assumed distribution (here logistic) to find the "most likely" values of the parameters to produce the data we see. In fact, it can be shown mathematically that the OLS solution in linear regression is the same as the MLE for linear regression. The likelihood function measures how probable a specific grid of β values is to have produced your data, so we want to MAXIMIZE that as seen in the plot below.



Maximum Likelihood Function Example

Let's see how to run binary logistic regression in each of our softwares!

R Python SAS

The `glm` function in R will provide us the ability to model binary logistic regressions. Similar to the `lm` function, you can specify a model formula. Notice the `factor` function on the *Central_Air* variable. This will convert this categorical variable into usable design variables for the modeling function.

The `family = binomial(link = "logit")` option is there to specify that we are building a logistic model. Generalized linear models (GLM) are a general class of models where logistic regression is a special case where the link function is the logit function. Use the `summary` function to look at the necessary output.

▼ Code

```
logit.model <- glm(Bonus ~ Gr_Liv_Area + factor(Central_Air),
                   data = train, family = binomial(link = "logi
summary(logit.model)
```

```
Call:
glm(formula = Bonus ~ Gr_Liv_Area + factor(Central_Air), family
    data = train)

Coefficients:
                      Estimate Std. Error z value Pr(>|z|)    
(Intercept)         -1.035e+01  6.422e-01  -16.12  < 2e-16 ***
Gr_Liv_Area          4.112e-03  1.962e-04   20.96  < 2e-16 ***
factor(Central_Air)Y  3.952e+00  5.180e-01    7.63 2.35e-14 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

    Null deviance: 2775.8  on 2050  degrees of freedom
Residual deviance: 1808.8  on 2048  degrees of freedom
AIC: 1814.8

Number of Fisher Scoring iterations: 6
```

Let's examine the output above. Scanning down the output, you can see the actual logistic regression equation itself for each of the variables, including the design variable created for *Central_Air* with the `factor` function. At our significance level (we used an 0.005 significance level for this analysis based on the sample size) it appears that both of these variables are significant. We will address variable selection in the "Subset Selection and Diagnostics" section.

To perform a global Likelihood Ratio Test to see if any of the variables are significant overall you can run the following code.

▼ Code

```
logit.model.r <- glm(Bonus ~ 1,
                     data = train, family = binomial(link = "lo

anova(logit.model, logit.model.r, test = 'LRT')
```

```
Analysis of Deviance Table

Model 1: Bonus ~ Gr_Liv_Area + factor(Central_Air)
Model 2: Bonus ~ 1
  Resid. Df Resid. Dev Df Deviance  Pr(>Chi)    
1      2048     1808.8                          
2      2050     2775.8 -2  -966.96 < 2.2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

The above code creates a second model without any variables. It then performs a Likelihood Ratio Test (LRT) on this model compared to the original model. The results of this test show the usefulness of all the variables. This is comparable to the global F-test in linear regression. The two models are significantly different (based on our significance level) which implies that at least one of the variables is significant. If these models weren't significantly different then the variables would be adding no value statistically.

For the *Central_Air* variable, we only have one design variable, but if our variable had 3 or more categories, we would have multiple design variables. Categorical variables should not be evaluated solely based on their individual categorical design variable significance as those represent specific tests between combinations of categories. The variable overall must be evaluated for its usefulness to the model. We will produce that with the next piece of code below to check the *Fireplaces* variable.

▼ Code

```
logit.model.f <- glm(Bonus ~ Gr_Liv_Area + factor(Central_Air) 
                     data = train, family = binomial(link = "lo

car::Anova(logit.model.f, test = 'LR', type = 'III')
```

```
Analysis of Deviance Table (Type III tests)

Response: Bonus
                    LR Chisq Df Pr(>Chisq)    
Gr_Liv_Area           565.89  1  < 2.2e-16 ***
factor(Central_Air)    86.81  1  < 2.2e-16 ***
factor(Fireplaces)     62.61  4  8.181e-13 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

The above code creates a second model with the *Fireplaces* categorical variable. It then uses a different `Anova` function from the `car` package (notice the capital A in `Anova`). This function performs a LRT comparing a model with and without a variable but leaving every other variable in there. Notice the options for `test = 'LR'` to get the LRT as well as the `type = 'III'` to get the Type 3 LRTs. This just means that the test only drops that specific variable when comparing to the full model. The results of this test show the usefulness of the *Fireplaces* variable. The two models are significantly different (based on our significance level) which implies that the *Fireplaces* variable (the only difference between comparing a model with and without the *Fireplaces* variable) is significant as a whole. If these models weren't significantly different then the *Fireplaces* variable would be adding no value statistically.

Lastly we can get the odds ratios for each of the above mentioned variables with the following code:

▼ Code

```
exp(
  cbind(coef(logit.model), confint(logit.model))
  )
```

```
                                     2.5 %       97.5 %
(Intercept)          3.184558e-05 8.233966e-06 1.041852e-04
Gr_Liv_Area          1.004121e+00 1.003745e+00 1.004517e+00
factor(Central_Air)Y 5.203450e+01 2.058035e+01 1.620722e+02
```

Odds ratios are a great way of interpreting the output from a logistic regression. For example, let's look at the *Central_Air* variable odds ratio. This tells us that homes with central air are 52 times as likely (in terms of odds) of being bonus eligible, on average, then homes without central air. To practice interpreting continuous variables, let's look at the *Gr_Liv_Area* variable. Every additional square foot of living area leads to the home being 1.004 times as likely to be bonus eligible. This maybe a little harder to easily understand. Another way to put this would be that every additional square foot of living area increases the odds of the home being bonus eligible by 0.4% (100*(1.004 - 1)) on average.

Testing Assumptions

Outside of independence of observations, the biggest assumption of logistic regression is that the continuous predictor variables are linearly related to the logit function (our transformation of the probability of our target). A great way to check this assumption is through Generalized Additive Models (GAMs).

Generalized additive models (GAM's) can be used to help evaluate the linearity of the relationship between the continuous predictor variables and the logit. When GAM's are applied to a logistic regression, the following is the new model:

$$\log(\frac{p_i}{1 - p_i}) = \beta_0 + f_1(x_1) + \cdots + f_k(x_k)$$

These functions applied to the continuous predictor variables need to be estimated. GAM's use spline functions to estimate these. If these splines say a straight line is good, then the assumption is met. There are some options if the assumption is not met:

1. Use the GAM representation of the logistic regression model instead of the traditional logistic regression model.
2. Strategically bin the continuous predictor variable.

In the first approach, instead of using the logistic regression model as previously defined, the logistic GAM would be used for predictions. The GAM version of the logistic regression is less interpretable in the traditional sense of odds ratios. Instead, plots are used to show potentially complicated relationships.

In the second approach, the continuous variables are categorized and put in the original logistic framework in their categorical form. There are statistical approaches to doing this, but one could also use the GAM plots to decide on possible splits for binning the data.

Let's see how to produce GAMs in each of our softwares!

R Python SAS

For GAMs we can use the `gam` function in R. Similar to the `glm` function, you have a model formula to specify the full model. Categorical variables can still be handled with the `factor` function. The only difference is that we will test the continuous variables with splines to see if those splines produce anything significantly better than their linear representations. The `s` function is put on every one of the continuous variables. The `family =` and `link =` options are used to specify the logistic regression. The `method =` option is specified here as their are multiple approaches to fitting the GAM in R. We will use the `REML` approach.

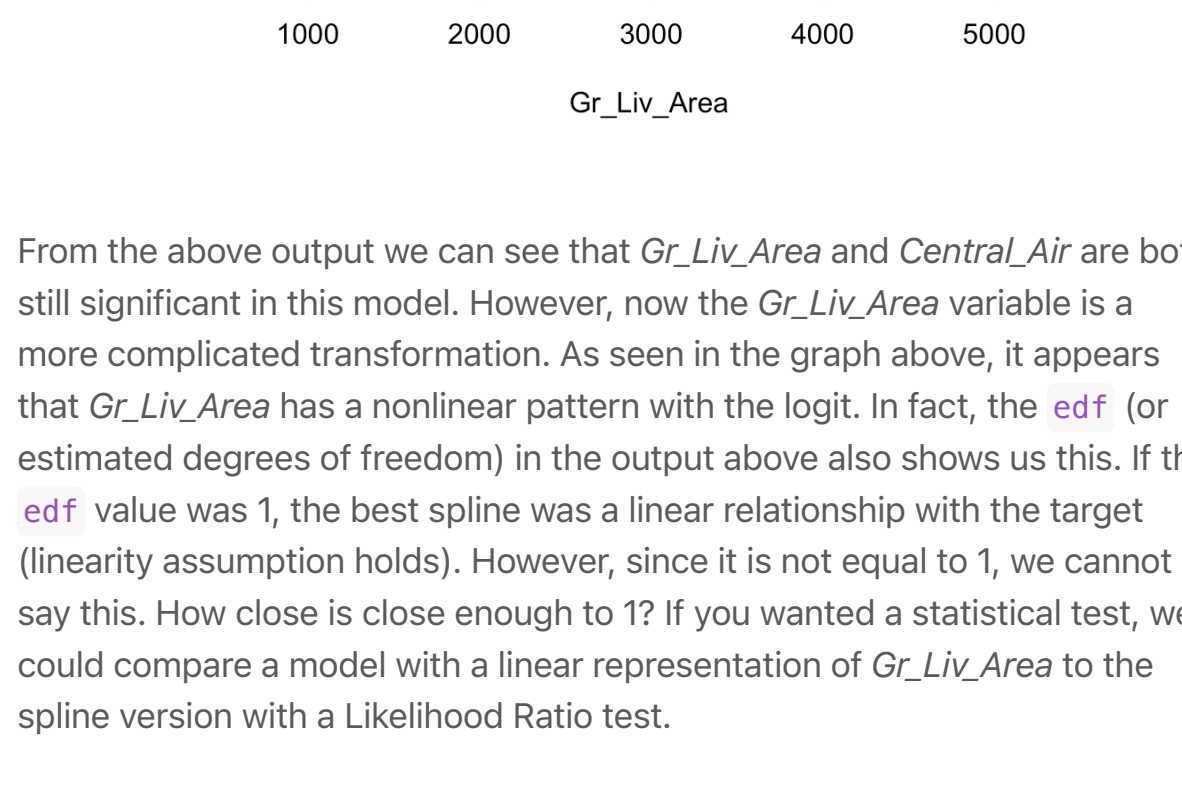
▼ Code

```
library(mgcv)

fit.gam <- mgcv::gam(Bonus ~ s(Gr_Liv_Area) + factor(Central_Ai
                     data = train, family = binomial(link = 'logit'),
                     method = 'REML')
summary(fit.gam)
```

```
Family: binomial 
Link function: logit 

Formula:
Bonus ~ s(Gr_Liv_Area) + factor(Central_Air)

Parametric coefficients:
                     Estimate Std. Error z value Pr(>|z|)    
(Intercept)           -4.4616     0.5033  -8.864  < 2e-16 ***
factor(Central_Air)Y   3.4882     0.4911   7.103 1.22e-12 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Approximate significance of smooth terms:
                 edf Ref.df Chi.sq p-value    
s(Gr_Liv_Area) 6.221  7.232  380.4  <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

R-sq.(adj) =   0.43   Deviance explained =   39%
-REML = 859.46  Scale est. = 1         n = 2051
```

▼ Code

```
plot(fit.gam)
```



From the above output we can see that *Gr_Liv_Area* and *Central_Air* are both still significant in this model. However, now the *Gr_Liv_Area* variable is a more complicated transformation. As seen in the graph above, it appears that *Gr_Liv_Area* has a nonlinear pattern with the logit. In fact, the `edf` (or estimated degrees of freedom) in the output above also shows us this. If the `edf` value was 1, the best spline was a linear relationship with the target (linearity assumption holds). However, since it is not equal to 1, we cannot say this. How close is close enough to 1? If you wanted a statistical test, we could compare a model with a linear representation of *Gr_Liv_Area* to the spline version with a Likelihood Ratio test.

▼ Code

```
anova(logit.model, fit.gam, test="LRT")
```

```
Analysis of Deviance Table

Model 1: Bonus ~ Gr_Liv_Area + factor(Central_Air)
Model 2: Bonus ~ s(Gr_Liv_Area) + factor(Central_Air)
  Resid. Df Resid. Dev     Df Deviance  Pr(>Chi)    
1    2048.0     1808.8                              
2    2042.8     1692.3 5.2212   116.58 < 2.2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

The null hypothesis to the above test is that the two models are equal. This null hypothesis would imply that the linearity assumption would hold since the spline is not estimating anything more complicated. However, the p-value was extremely small and therefore the two models are *not* equal to other. This means that *Gr_Liv_Area* cannot just be modeled linearly with the logit.

From the plot above, it appears that *Gr_Liv_Area* has three main patterns - below 1300 sqft, between 1300 and 4000 sqft, and above 4000 sqft. But we want bins that are about the same in their effect. Let's create a new variable along the cut points of < 1000, 1000-1500, 1500-3000, 300-4500, 4500+.

▼ Code

```
train <- train %>%
  mutate(Gr_Liv_Area_BIN = cut(Gr_Liv_Area, breaks = c(-Inf,100
```

Now let's rebuild the logistic regression model with this new categorical version of the *Gr_Liv_Area* variable.

▼ Code

```
logit.model.bin <- glm(Bonus ~ factor(Gr_Liv_Area_BIN) + factor
                       data = train, family = binomial(link = '

summary(logit.model.bin)
```

```
Call:
glm(formula = Bonus ~ factor(Gr_Liv_Area_BIN) + factor(Central_A
    family = binomial(link = "logit"), data = train)

Coefficients:
                                      Estimate Std. Error z val
(Intercept)                            -8.8210     1.1065  -7.9
factor(Gr_Liv_Area_BIN)(1e+03,1.5e+03]   4.5121     1.0052   4.4
factor(Gr_Liv_Area_BIN)(1.5e+03,3e+03]   6.6437     1.0049   6.6
factor(Gr_Liv_Area_BIN)(3e+03,4.5e+03]  21.1646   363.8508   0.0
factor(Gr_Liv_Area_BIN)(4.5e+03, Inf]    5.5986     1.7331   3.2
factor(Central_Air)Y                     3.2224     0.4734   6.8
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

    Null deviance: 2775.8  on 2050  degrees of freedom
Residual deviance: 1892.0  on 2045  degrees of freedom
AIC: 1904

Number of Fisher Scoring iterations: 14
```

From the above output, we can see that *Gr_Liv_Area_BIN* is still a significant variable in our model. The nice part about using a categorical representation for the variable instead of using the GAM for predictions is that we still have some interpretability on this variable using odds ratios for the categories.

Predicted Values

Obviously, the predicted logit values really don't help us too much. Instead we want to gather the predicted probabilities of the target variable categories. Luckily, the software we are looking at make this rather easy to do.

Let's see how to produce these predicted probabilities in each of our softwares!

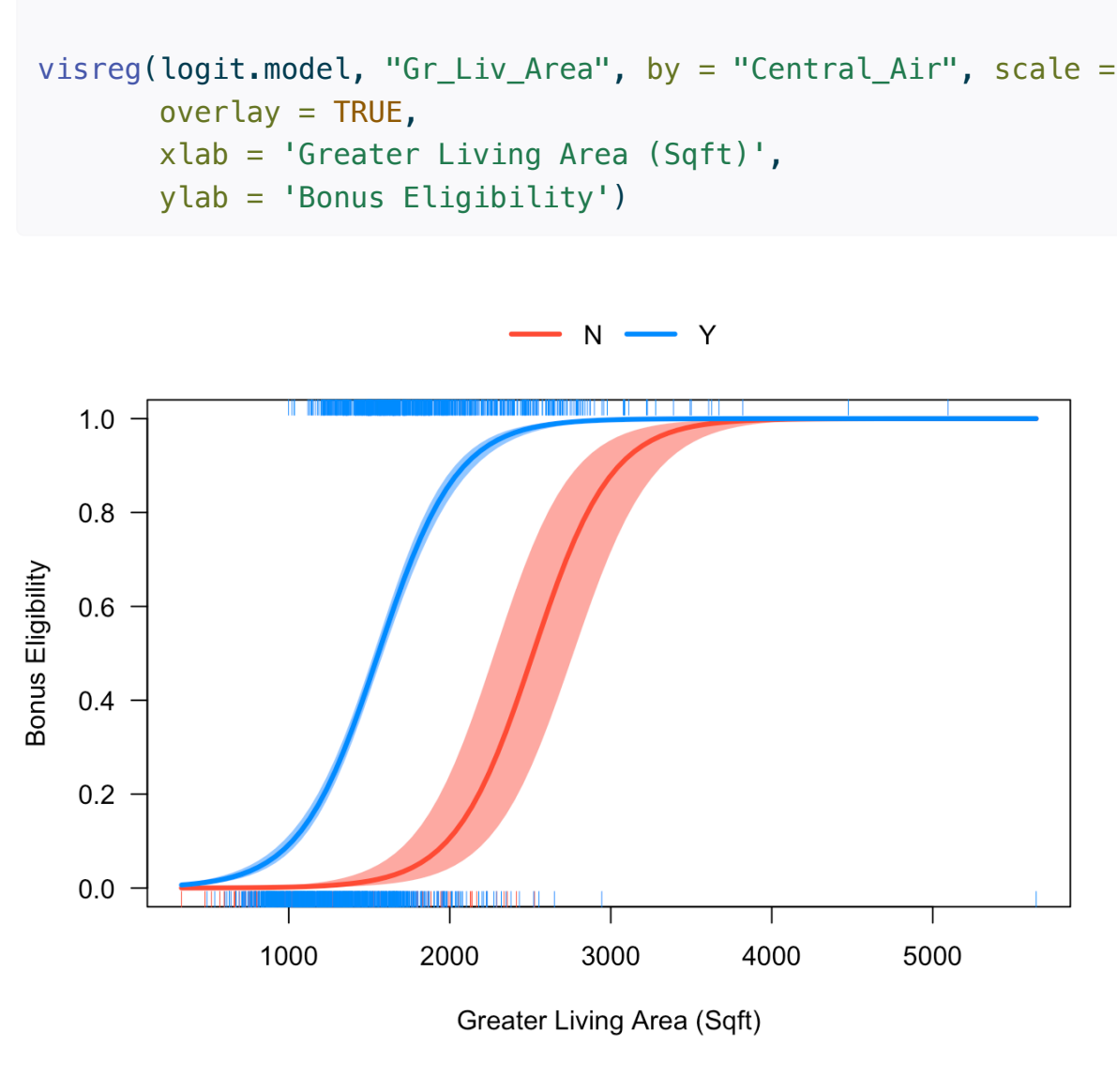
R Python SAS

If you split your data into training and validation (and/or testing) you could score that data set using the `predict` function. Here we are creating a new data set of 5 observations to score instead. The `predict` function has three primary pieces - the model object (here called `logit.model`, the `newdata =` option, and the `type =` option. The `newdata =` option is where you specify the data set you want scored, while the `type =` option specifies the type of prediction you want. Here we want the `response` option since we are predicting the probabilities. If we wanted the predicted logit values we could use the option `terms` instead.

The `visreg` function below creates the effect plot, which will show us the logistic curve across different values of the continuous variable, here *Gr_Liv_Area*. It will show each of the categorical variable categories as a separate curve. If there is were more continuous variables they are just taken at a specified value - their average value.

▼ Code

```
new_ames <- data.frame(Gr_Liv_Area = c(1500, 2000, 2250, 2500, 
                       Central_Air = c("N", "Y", "Y", "N", "Y")

new_ames <- data.frame(new_ames, 'Pred' = predict(logit.model, 

library(visreg)

visreg(logit.model, "Gr_Liv_Area", by = "Central_Air", scale = 
       overlay = TRUE,
       xlab = 'Greater Living Area (Sqft)',
       ylab = 'Bonus Eligibility')
```



▼ Code

```
print(new_ames)
print(new_ames)
```

```
  Gr_Liv_Area Central_Air       Pred
1        1500           N 0.01498152
2        2000           Y 0.86084436
3        2250           Y 0.94534188
4        2500           N 0.48167577
5        3500           Y 0.99966165
```

The `print` function above looks at the scored data set. Notice that we added an additional column to our original data set we were scoring. This is the predicted probability of a 1. This is default in R. If you want the predicted probability of the 0 category, you can just subtract this prediction from 1.

← Categorical Data Analysis
Data Considerations →